



RentNest

ভাড়া বাসা খোঁজা ও ভাড়া দেওয়ার সম্পূর্ণ REST API

Node.js + Express 5

TypeScript

PostgreSQL + Prisma 7

Stripe

Assignment 4 — Backend Project · B7A4

সমস্যা এবং সমাধান

এই API কী কাজ করে

সমস্যা

ভাড়া বাসা খোঁজা এখনো ছড়ানো-ছিটানো — বিজ্ঞাপন এক জায়গায়, দরদাম আরেক জায়গায়, টাকা লেনদেনের কোনো নিরাপদ রেকর্ড নেই। বাড়িওয়ালা কাকে বাসা দিচ্ছেন জানেন না, ভাড়াটিয়াও জানেন না বাসাটা আসল কিনা।

সমাধান

- ▶ একটাই প্ল্যাটফর্মে **লিস্টিং** → **রিকোয়েস্ট** → **অনুমোদন** → **পেমেন্ট** → **রিভিউ**
- ▶ প্রতিটা ধাপ ডেটাবেসে **status** হিসেবে ট্র্যাক করা
- ▶ **Stripe** দিয়ে আসল পেমেন্ট, ভুয়া নয়
- ▶ Admin পুরো প্ল্যাটফর্ম নজরে রাখেন

এক নজরে

প্রজেক্টের পরিসর

সংখ্যায় RentNest

9

Feature Module

6

Database Table

30+

API Endpoint

3

ইউজার Role

27

Git Commit

100%

TypeScript

প্রতিটা module আলাদা ফোল্ডারে — `route → controller → service → validation` — তাই নতুন ফিচার যোগ করলেও পুরনো কোড ছুঁতে হয় না।

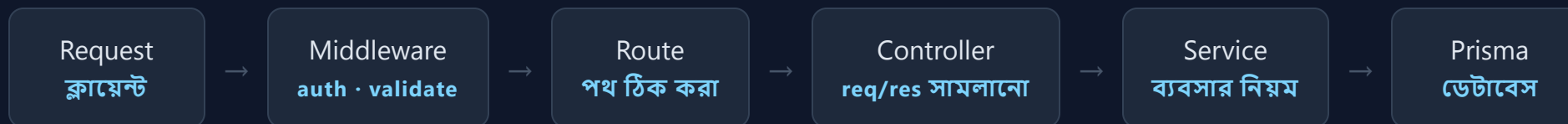
কী দিয়ে বানানো এবং কেন

প্রতিটা টুল বেছে নেওয়ার পেছনে কারণ আছে

স্তর	টেকনোলজি	কেন এটাই
Runtime	Node.js 20+	Vercel-এ সরাসরি চলে, ইকোসিস্টেম বড়
Framework	Express 5	async error স্বয়ংক্রিয়ভাবে ধরে, middleware সহজ
ভাষা	TypeScript	বাগ রানটাইমে নয়, লেখার সময়েই ধরা পড়ে
ডেটাবেস	PostgreSQL	রিলেশন আর Decimal টাকা-পয়সার জন্য নিরাপদ
ORM	Prisma 7	schema থেকেই টাইপ তৈরি হয়, migration সহজ
Auth	JWT	httpOnly cookie + Bearer — দুইভাবেই চলে
Validation	Zod 4	schema-ই টাইপ, আলাদা করে লিখতে হয় না
পেমেন্ট	Stripe	Checkout + Webhook, আসল লেনদেন
সিকিউরিটি	helmet · cors · rate-limit · bcrypt	সাধারণ আক্রমণগুলো আগেই ঠেকানো

একটা রিকোয়েস্ট যে পথে যায়

প্রতিটা স্তরের একটাই দায়িত্ব



Middleware

`auth("ADMIN")` টোকেন যাচাই করে role মেলায়। `validateRequest` Zod দিয়ে ইনপুট পরীক্ষা করে — ভুল ডেটা কখনো service পর্যন্ত পৌঁছায় না।

Service

সব সিদ্ধান্ত এখানে — কে কী দেখতে পাবে, status কখন বদলাবে। Controller-এ কোনো ব্যবসার নিয়ম নেই, তাই পরীক্ষা করা সহজ।

Error

একটাই `globalErrorHandler`। Zod, Prisma, JWT — সব ভুল একই কাঠামোয় ফেরত যায়, কোথাও আলাদা করে `try/catch` লিখতে হয় না।

৬টা টেবিল এবং তাদের সম্পর্ক

Prisma schema থেকে সরাসরি

টেবিল	কী রাখে	সম্পর্ক
User	নাম, ইমেইল, hash করা পাসওয়ার্ড, role, status	একজন User → অনেক Property / Rental / Payment
Category	বাসার ধরন (Apartment, Studio...), slug, isActive	এক Category → অনেক Property
Property	ঠিকানা, ভাড়া, bedroom, amenities, images, status	Landlord ও Category-র সাথে যুক্ত
RentalRequest	move-in তারিখ, quotedAmount, status, সময়ের স্ট্যাম্প	Property + Tenant + Landlord — তিনটাকে জোড়া লাগায়
Payment	transactionId, amount, provider, status, paidAt	এক RentalRequest → অনেক Payment (retry-র জন্য)
Review	rating, comment, isVisible	এক Rental → একটাই Review @unique

খেয়াল করার মতো: টাকার জায়গায় `Decimal(10,2)` — float নয়, তাই এক পয়সাও হারায় না। আর `onDelete: Restrict` দেওয়া, তাই পেমেন্ট থাকা অবস্থায় কেউ ভুল করে ডেটা মুছে ফেলতে পারে না।

তিনটা Role, তিন রকম ক্ষমতা

রেজিস্ট্রেশনের সময় ইউজার নিজেই role বেছে নেন — শুধু ADMIN বাদে

TENANT

- ▶ বাসা খোঁজা ও ফিল্টার করা
- ▶ ভাড়ার রিকোয়েস্ট পাঠানো
- ▶ অনুমোদনের পর **পেমেন্ট**
- ▶ রিকোয়েস্ট বাতিল করা
- ▶ ভাড়া শেষে **রিভিউ** দেওয়া

LANDLORD

- ▶ নিজের লিস্টিং তৈরি ও সম্পাদনা
- ▶ আসা রিকোয়েস্ট দেখা
- ▶ **Approve / Reject** করা
- ▶ ভাড়া শেষ হলে COMPLETED করা
- ▶ নিজের বাসার রিভিউ দেখা

ADMIN

- ▶ সব ইউজার দেখা, **ban / unban**
- ▶ সব property (ARCHIVED সহ)
- ▶ সব rental ও সব payment
- ▶ Category তৈরি ও পরিচালনা

নিরাপত্তার সীমারেখা: ADMIN পাবলিক রেজিস্ট্রেশনে তৈরি করাই যায় না — শুধু seed script বানায়। আর কোনো admin নিজেকে বা অন্য admin-কে ban করতে পারে না।

প্রধান এন্ডপয়েন্টগুলো

৩০-এর বেশি endpoint, ৯টা module-এ ভাগ করা

Auth ও Profile

- ▶ POST /api/auth/register
- ▶ POST /api/auth/login
- ▶ GET /api/auth/me
- ▶ PATCH /api/users/me/password

Property (পাবলিক)

- ▶ GET /api/properties
- ▶ GET /api/properties/:id
- ▶ search · city · price · bedrooms · sort · page

Rental ও Landlord

- ▶ POST /api/rentals
- ▶ PATCH /api/landlord/requests/:id
- ▶ PATCH /api/rentals/:id/cancel

Payment ও Admin

- ▶ POST /api/payments/create
- ▶ POST /api/payments/confirm
Stripe webhook
- ▶ GET /api/admin/users
- ▶ PATCH /api/admin/users/:id

একটা ভাড়ার পুরো যাত্রা

প্রতিটা তীরই ডেটাবেসে একটা status পরিবর্তন



🔒 প্রতিটা ধাপে পাহারা

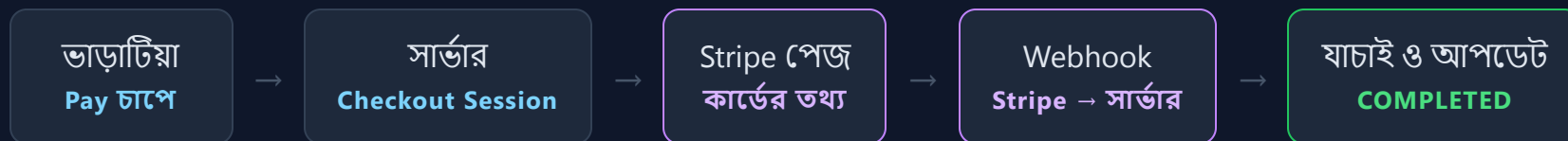
- ▶ APPROVED না হলে পেমেন্টই শুরু করা যায় না
- ▶ COMPLETED না হলে রিভিউ দেওয়া যায় না
- ▶ এক rental-এ একটাই রিভিউ — ডেটাবেস নিজেই ঠেকায়

🔄 অন্য পথগুলো

- ▶ বাড়িওয়ালা মানা করলে → **REJECTED**
- ▶ ভাড়াটিয়া সরে গেলে → **CANCELLED**
- ▶ পেমেন্ট ব্যর্থ হলে → **FAILED**, আবার চেষ্টা করা যায়

Stripe দিয়ে আসল লেনদেন

ব্রাউজারের কথা বিশ্বাস করা হয় না — Stripe নিজে যা বলে, সেটাই মানা হয়



💡 কেন Webhook

ভাড়াটিয়া টাকা দিয়ে ব্রাউজার বন্ধ করে ফেললেও পেমেন্ট হারায় না — Stripe সরাসরি সার্ভারকে জানায়। আর কেউ যদি নিজে থেকে `success` URL-এ ঢুকে পড়ে, তাতেও কিছু হয় না, কারণ Stripe না বললে status বদলায় না।

📁 যা রেকর্ড থাকে

- ▶ নিজস্ব `transactionId` (unique)
- ▶ Stripe-এর session ও intent আইডি
- ▶ `amount · currency · method · paidAt`
- ▶ পুরো ঘটনার `metadata` — অডিটের জন্য

যে সমস্যাটা সবচেয়ে কঠিন ছিল

Serverless-এ Stripe webhook যাচাই করা

❌ সমস্যা

Stripe webhook যাচাই করতে **হুবহু raw body** লাগে — signature ওই কাঁচা বাইটের উপর হিসাব করা।

কিন্তু Vercel-এর মতো serverless প্ল্যাটফর্ম কোড চলার **আগেই** body-টা JSON-এ পার্স করে ফেলে। ফলে raw বাইট আর পাওয়া যায় না, আর `constructEvent()` সবসময় ব্যর্থ হয়।

লোকালে সব ঠিক চলে — শুধু production-এ ভাঙে। এই ধরনের বাগ ধরা সবচেয়ে কঠিন।

✅ সমাধান

যাচাই করার **দুটো পথ** রাখলাম:

- **স্বাভাবিক পথ:** raw body পেলে signature দিয়ে যাচাই
- **Fallback:** raw body না পেলে event-এর আইডি নিয়ে **সরাসরি Stripe API থেকে event-টা আবার আনা হয়**

Stripe নিজে উত্তর দেয় বলে **জাল event এখানেও টেকে না** — নিরাপত্তা এক চুলও কমেনি, অথচ serverless-এও কাজ করে।

যা শিখলাম: লাইব্রেরি কী চায় শুধু সেটা নয়, সে কেন চায় সেটা বুঝলে বিকল্প পথ বের করা যায়।

ভ্যালিডেশন ও এরর হ্যান্ডলিং

ভুল ইনপুট যেন কখনো ডেটাবেস পর্যন্ত না পৌঁছায়

✓ সফল উত্তর

```
{
  "success": true,
  "statusCode": 200,
  "message": "Properties retrieved",
  "data": [ ... ],
  "meta": { "page": 1, "limit": 10, "total": 14 }
}
```

প্রতিটা সফল উত্তরের গঠন একই — frontend-কে কখনো অনুমান করতে হয় না।

⚠ ভুলের উত্তর

```
{
  "success": false,
  "statusCode": 400,
  "message": "Validation Error",
  "errorDetails": [
    { "path": "email", "message": "Invalid email" }
  ]
}
```

কোন ফিল্ডে কী ভুল, তা `path` ধরে বলা — ইউজারকে ঠিক জায়গায় লাল দাগ দেখানো যায়।

Zod, Prisma, JWT — সব রকম ভুল একটাই `globalErrorHandler` -এ গিয়ে এই একই ফরম্যাটে রূপ নেয়। ৪০৪ আর সার্ভার ক্র্যাশও একই কাঠামোয় ফেরত আসে।

যেভাবে সুরক্ষিত রাখা হয়েছে

একাধিক স্তরে প্রতিরক্ষা

ব্যবস্থা	কী ঠেকায়
bcrypt (১২ রাউন্ড)	পাসওয়ার্ড কখনো প্লেইন টেক্সটে থাকে না; ডেটাবেস ফাঁস হলেও পাসওয়ার্ড উদ্ধার করা প্রায় অসম্ভব
password omit	প্রতিটা query-তে পাসওয়ার্ড ফিল্ড বাদ দেওয়া — ভুল করেও API response-এ যাওয়ার পথ নেই
JWT + httpOnly cookie	টোকেন JavaScript থেকে পড়া যায় না, তাই XSS দিয়ে চুরি করা কঠিন
Role middleware	ভাড়াটিয়া landlord-এর URL জানলেও ঢুকতে পারে না — প্রতিটা route-এ role যাচাই হয়
Ownership যাচাই	শুধু লগইন যথেষ্ট নয় — id জানলেই অন্যের rental বা payment দেখা যায় না
express-rate-limit	একই IP থেকে বারবার লগইন চেষ্টা করে পাসওয়ার্ড অনুমান করা ঠেকায়
helmet · cors	নিরাপত্তা হেডার যোগ করে, আর শুধু অনুমোদিত origin থেকেই কল গ্রহণ করে
Zod validation	অপ্রত্যাশিত বা ক্ষতিকর ইনপুট service স্তরে পৌঁছানোর আগেই আটকে যায়

যে ধাপগুলো দেখাব

Postman দিয়ে সরাসরি

▶ ডেমোর ক্রম

- ▶ ১. Health check — সার্ভার ও ডেটাবেস চালু
- ▶ ২. ভাড়াটিয়া হিসেবে রেজিস্টার ও লগইন
- ▶ ৩. ফিল্টার দিয়ে বাসা খোঁজা
- ▶ ৪. ভাড়ার রিকোয়েস্ট পাঠানো
- ▶ ৫. বাড়িওয়ালা হিসেবে approve করা
- ▶ ৬. Stripe দিয়ে পেমেন্ট
- ▶ ৭. Admin হিসেবে সবকিছু দেখা ও ban করা
- ▶ ৮. ভুল ইনপুট দিয়ে error দেখানো

🔑 ডেমো অ্যাকাউন্ট

ADMIN	admin@rentnest.com
LANDLORD	demo.landlord@rentnest.com
TENANT	demo.tenant@rentnest.com

সব ডেমো ডেটা `npm run seed` দিয়ে তৈরি — ক্যাটাগরি, ইউজার আর নমুনা বাসাসহ।

ডিপ্লয়মেন্ট

লাইভ এবং চলমান

Vercel-এ ডিপ্লয় করা, GitHub-এর সাথে যুক্ত

আইটেম	লিংক
Live API	<code>https://assingment4-nu.vercel.app</code>
GitHub Repo	<code>github.com/MDboni/assingment4</code>
Health Check	<code>/health</code> — ডেটাবেস সংযোগসহ যাচাই করে

CI/CD

`main` -এ push করলেই স্বয়ংক্রিয় ডিপ্লয়।
অন্য branch-এ push করলে আলাদা
preview লিংক তৈরি হয়।

ডেটাবেস

ক্লাউড PostgreSQL, connection pooling
সহ। Migration চলে `prisma migrate`
`deploy` দিয়ে।

Secrets

কোনো key কোডে নেই — সব Vercel-এর
environment variable-এ রাখা।



ধন্যবাদ

RentNest — একটা সম্পূর্ণ rental marketplace API, যেখানে ভাড়ার রিকোয়েস্ট থেকে আসল পেমেন্ট পর্যন্ত পুরো পথটা নিরাপদে ট্র্যাক করা হয়।

৩টা Role

৬টা টেবিল

৩০+ Endpoint

Stripe পেমেন্ট

TypeScript

প্রশ্ন থাকলে করুন 🙋